

手把手之用户如何实现分库分表

分库分表介绍

很多情况下，分库分表并不是从系统设计开始就存在的，而是系统运行过程中，出现数据量庞大或者查询性能慢等问题延伸而来。

什么是分库分表

从原理上说，分库是将原本的单库拆分为多个库，分表是将原来的单表拆分为多个表。

分库有两种模式：

- 垂直拆库：电商库 MallDB，业务拆分后就是 UserDB、OrderDB、PayDB...
- 分片拆库：用户库 UserDB，分片库后就是 UserDB_0、UserDB_1、UserDB_xx

分表也有两种模式：

- 垂直拆分：订单表 OrderTable，拆分后就是 OrderTable 以及 OrderExtTable。
- 水平拆分：订单表 OrderTable，拆分后就是 OrderTable_0、OrderTable_xxx。

什么场景下进行分库分表

1) 什么场景下分表？数据量过大或者数据库表对应的磁盘文件过大。

Q：多少数据量进行分表？

A：单表 1000w 是否要分表？回答不够标准。

假设一个表里 15 个字段，没有特别大的值（不包含 text 或其它超长度的列）数据量超过 5000 万了，依然很丝滑，因为走索引。

真正需要考虑的是：业务的增长量以及历史数量。

Q：物理文件过大，会有什么问题？

A: 会影响公司对数据库表的一个备份。数据库表文件过大, 也间接证明表数据过大, 增加或删除字段导致锁表的时间过长。

2) 什么场景下分库?

当数据库的连接不够客户端使用时, 可以考虑分库或读写分离。

如果说当数据库的 QPS 越来越高以及数据量越来越大的时候, 就需要考虑分库分表。

Q: 为什么说连接不够用?

A: 假设 MySQL Server 能支持 4000 个数据库连接。我们有 10 个服务, 40 个节点, 一个节点呢数据库连接池最多 10 个。这样就把一个 MySQL Server 的连接数压榨干净了。

当 MySQL 连接不够用时, 可能会报错 "Too many connections" 或者类似的错误。这是因为 MySQL 服务器同时可以处理的连接数量是有限制的, 当连接数达到这个限制时, 服务器就会拒绝新的连接请求, 并返回这个错误消息。

3) 什么场景下分库又分表?

- 高并发写入场景: 当应用面临高并发的写入请求时, 单一数据库可能无法满足写入压力, 此时可以将数据按照一定规则拆分到多个数据库中, 每个数据库处理部分数据的写入请求, 从而提高写入性能。
- 数据量巨大场景: 随着数据量的不断增加, 单一数据库的存储和查询性能可能逐渐下降。此时, 可以将数据按照一定的规则拆分到多个表中, 每个表存储部分数据, 从而分散数据的存储压力, 提高查询性能。

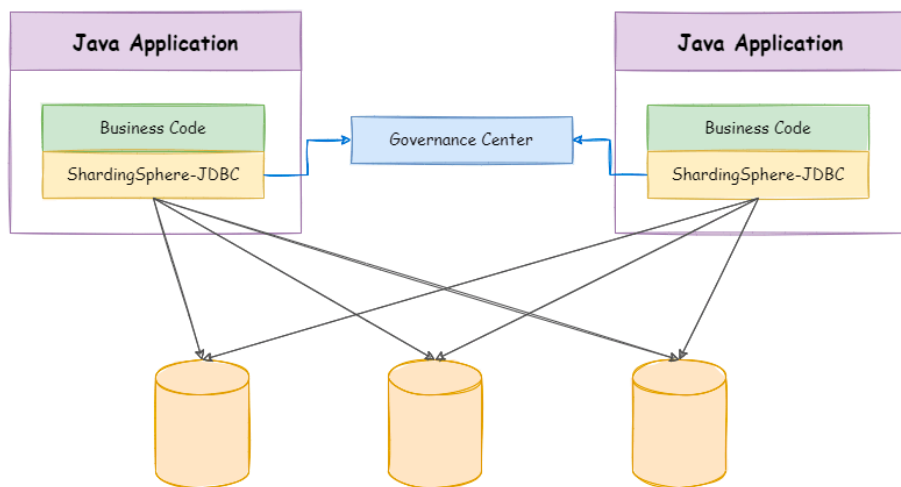
分库分表组件

ShardingSphere-JDBC

定位为轻量级 Java 框架, 在 Java 的 JDBC 层提供的额外服务。它使用客户端直连数据库, 以 jar 包形式提供服务, 无需额外部署和依赖, 可理解为增强版的 JDBC 驱动, 完全兼容 JDBC 和各种 ORM 框架。

- 适用于任何基于 JDBC 的 ORM 框架, 如: JPA, Hibernate, Mybatis, Spring JDBC Template 或直接使用 JDBC;
- 支持任何第三方的数据库连接池, 如: DBCP, C3P0, BoneCP, HikariCP 等;

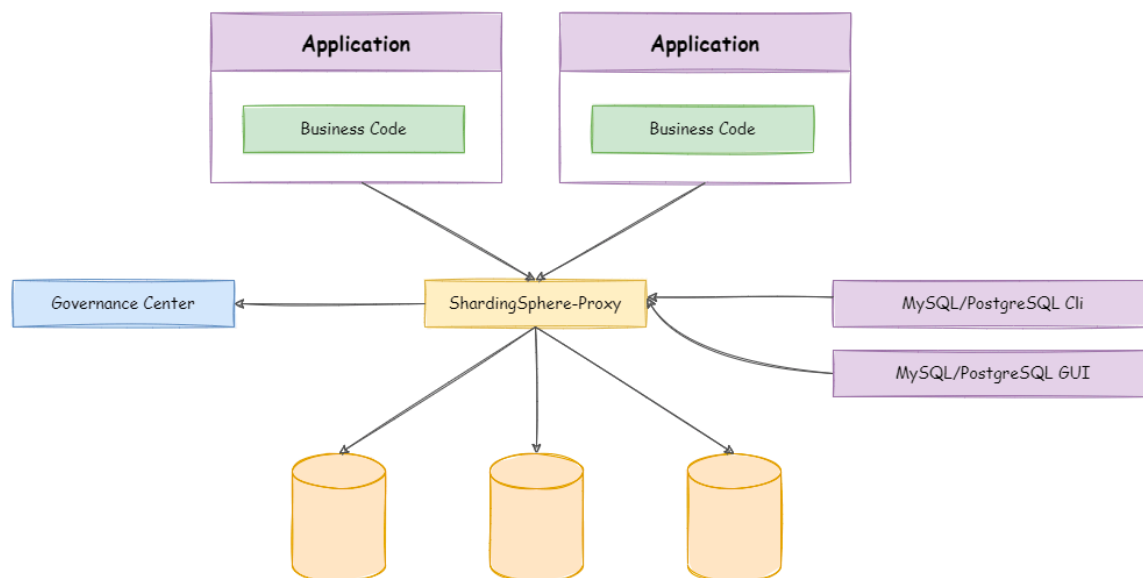
- 支持任意实现 JDBC 规范的数据库，目前支持 MySQL, PostgreSQL, Oracle, SQLServer 以及任何可使用 JDBC 访问的数据库。



ShardingSphere-Proxy

定位为透明化的数据库代理端，通过实现数据库二进制协议，对异构语言提供支持。目前提供 MySQL 和 PostgreSQL 协议，透明化数据库操作，对 DBA 更加友好。

- 向应用程序完全透明，可直接当做 MySQL/PostgreSQL 使用；
- 兼容 MariaDB 等基于 MySQL 协议的数据库，以及 openGauss 等基于 PostgreSQL 协议的数据库；
- 适用于任何兼容 MySQL/PostgreSQL 协议的客户端，如：MySQL Command Client, MySQL Workbench, Navicat 等。



JDBC 和 Proxy 优劣势

- JDBC:

优点：性能较高，通过 JDBC 直接向 MySQL 发起请求调用。优点：使用较为简单，理论上无需修改代码，仅需使用 ShardingSphere 的配置即可。缺点：需要修改项目配置以及引入 Jar 包。缺点：对应用的内存有一定影响。

- Proxy:

优点：无需对现有项目做任何配置或代码变更，将数据库的地址改为 Proxy 的地址即可。优点：Proxy 对 Java 应用内存没有任何影响。优点：分片后无法知道一条数据记录到底在那张表，Proxy 是屏蔽了分片逻辑，可直接操作逻辑表。缺点：JDBC 操作 MySQL 是点对点的，但是 Proxy 多了一层链路。

为什么选择 ShardingSphere?

- 拥有活跃的社区，能够提供及时的技术支持和更新；

- 代码质量极高，经过严格测试和验证，稳定可靠；
- 提供丰富的功能，满足多样化的业务需求。

用户分库分表策略

根据 2022 年的全国人口统计数据，现有 14 亿多总人口，每年新生人口约 956 万。为便于后续业务数据规模的判断，本文先基于这一人口总量和增长数据进行估算。

年末全国人口^[6]141175万人，

全年出生人口956万人，

表1 2022年年末人口数及其构成

指标	年末数（万人）	比重（%）
全国人口	141175	100.0
其中：城镇	92071	65.2
乡村	49104	34.8
其中：男性	72206	51.1
女性	68969	48.9
其中：0-15岁（含不满16周岁） ^[7]	25615	18.1
16-59岁（含不满60周岁）	87556	62.0
60周岁及以上	28004	19.8
其中：65周岁及以上	20978	14.9

根据系统设计的假设，12306 的注册用户规模约为 10 亿，每年新增用户约 1000 万。

在用户数据分库或分表之前，我们需要先考虑拆分成多少个库或表才能达到最优性能。为了进行这样的决策，我们可以预估单个表的最大数据量。根据过去的经验，通常我们会选择 2000 万作为一个经验值。这个数据量既不会过小，同时又能保证增删改查等操作相对流畅。

根据当前用户表的数据量为 10 亿，并且每年新增 1000 万用户，预估未来系统的生命周期较长，数据量大概会达到 30 亿左右。基于这个数据量，我们预估单表的数据量在 2000 万左右，因此需要分大约 150 张表来容纳这些数据。

在进行分库分表容量评估时，我们通常会尽可能多地进行评估。这样做的好处是，即使每张表的数据量不多，也能及早发现拆分后是否存在数据问题，以便及时进行调整和优化。

此外，需要特别指出的是，我们对表数据量考虑的阈值相对较小，这是因为我们的系统具备良好的可扩展性，可以轻松应对大量的数据增长。因此，基于这种情况的分库分表策略，即使在几百年后，这个分库分表依然能够处理数据，并且不会出现性能问题。这为我们的系统提供了稳定可靠的性能保障。

选择用户分片键

选择分库分表中的分片键（Sharding Key）是一个关键决策，它直接影响了分库分表的性能和可扩展性。以下是一些选择分片键的关键因素：

1. 访问频率：选择分片键应考虑数据的访问频率。将经常访问的数据放在同一个分片上，可以提高查询性能和降低跨分片查询的开销。
2. 数据均匀性：分片键应该保证数据的均匀分布在各个分片上，避免出现热点数据集中在某个分片上的情况。
3. 业务关联性：分片键应该与业务关联紧密，这样可以避免跨分片查询和跨库事务的复杂性。
4. 数据不可变：一旦选择了分片键，它应该是不可变的，不能随着业务的变化而频繁修改。

基于以上考虑，我们选择使用 username 作为分片键。

改造用户分库分表

如果你没有使用过 ShardingSphere 分库分表操作，可以查看官网进行一些前置条件理解。

数据分片 :: [ShardingSphere](#)

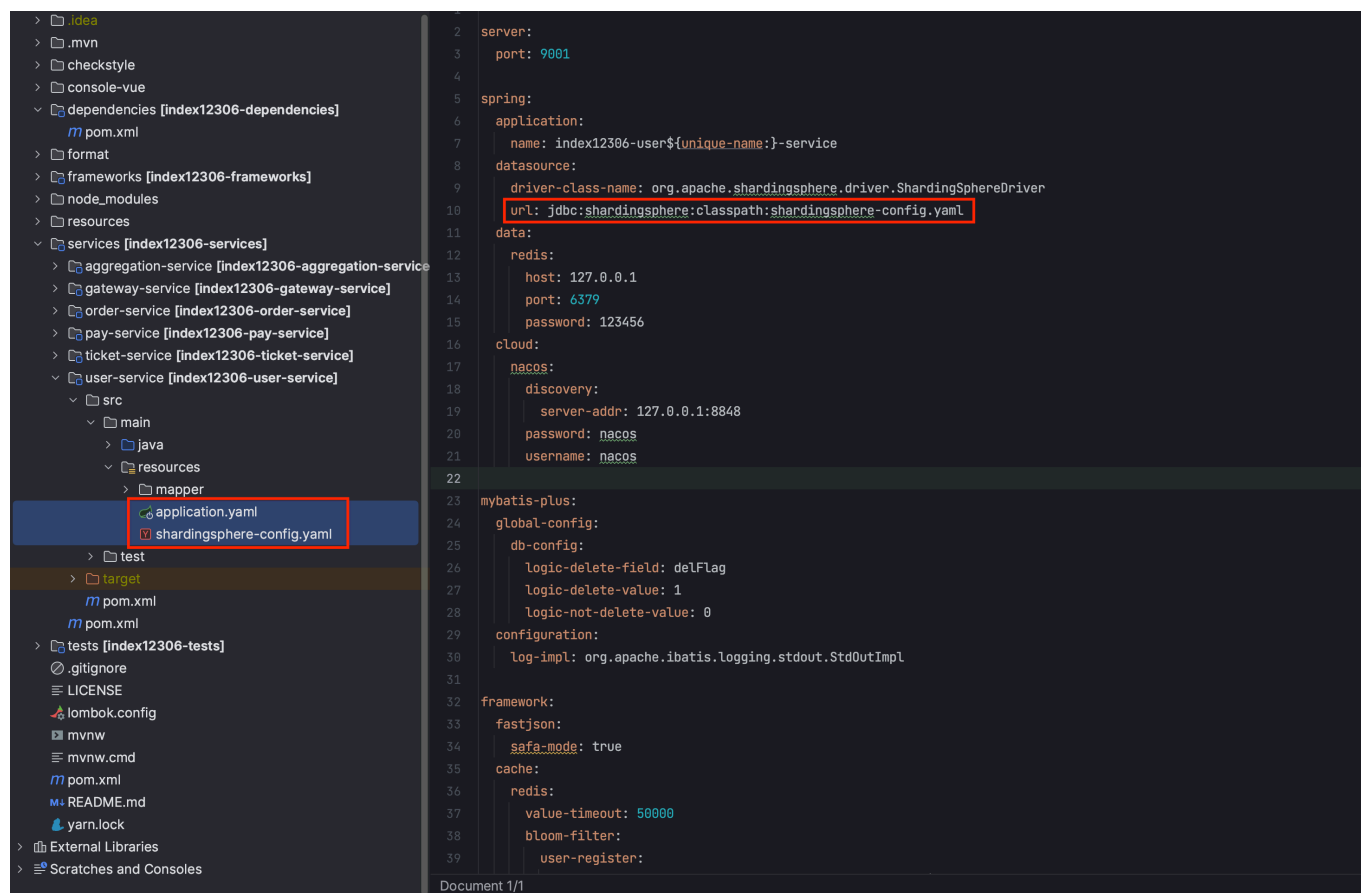
引入 ShardingSphere 依赖

```
<dependency>
  <groupId>org.apache.shardingsphere</groupId>
  <artifactId>shardingsphere-jdbc-core</artifactId>
  <version>5.3.2</version>
</dependency>
```

定义分片规则

```
spring:
  datasource:
    # ShardingSphere 对 Driver 自定义，实现分库分表等隐藏逻辑
    driver-class-name: org.apache.shardingsphere.driver.ShardingSphereDriver
    # ShardingSphere 配置文件路径
    url: jdbc:shardingsphere:classpath:shardingsphere-config.yaml
```

配置文件地址信息如下：



用户分片配置

因为 12306 更多的是向大家演示分库分表，所以分 2 个库以及对应业务 16 张表。

shardingsphere-config.yaml

```
# 数据源集合
datasources:
  ds_0:
    dataSourceClassName: com.zaxxer.hikari.HikariDataSource
    driverClassName: com.mysql.cj.jdbc.Driver
    jdbcUrl: jdbc:mysql://127.0.0.1:3306/12306_user_0?
    useUnicode=true&characterEncoding=UTF-
    8&rewriteBatchedStatements=true&allowMultiQueries=true&serverTimezone=Asia/Shanghai
    username: root
    password: root

  ds_1:
```

```

dataSourceClassName: com.zaxxer.hikari.HikariDataSource
driverClassName: com.mysql.cj.jdbc.Driver
jdbcUrl: jdbc:mysql://127.0.0.1:3306/12306_user_1?
useUnicode=true&characterEncoding=UTF-
8&rewriteBatchedStatements=true&allowMultiQueries=true&serverTimezone=Asia/Shanghai
username: root
password: root

rules:
- !SHARDING
  tables:
    t_user:
      # 真实数据节点，比如数据库源以及数据库在数据库中真实存在的
      actualDataNodes: ds_${0..1}.t_user_${0..15}
      # 分库策略
      databaseStrategy:
        # 用于单分片键的标准分片场景
        standard:
          # 分片键
          shardingColumn: username
          # 分片算法，对应 rules[0].shardingAlgorithms
          shardingAlgorithmName: user_database_hash_mod
      # 分表策略
      tableStrategy:
        # 用于单分片键的标准分片场景
        standard:
          # 分片键
          shardingColumn: username
          # 分片算法，对应 rules[0].shardingAlgorithms
          shardingAlgorithmName: user_table_hash_mod
      # 分片算法
      shardingAlgorithms:
        # 数据库分片算法
        user_database_hash_mod:
          # 根据分片键 Hash 分片
          type: HASH_MOD
          # 分片数量
          props:
            sharding-count: 2
        # 数据表分片算法
        user_table_hash_mod:

```



```
# 根据分片键 Hash 分片
type: HASH_MOD
# 分片数量
props:
  sharding-count: 16
# 展现逻辑 SQL & 真实 SQL
props:
  sql-show: true
```

写好以上配置后，就可以愉快使用数据库分库分表了。

`props.sql-show` 配置尽量打开，它能打印应用的逻辑 SQL 以及真正查询数据库的真实 SQL。

文末总结

分库分表是一种数据库架构设计和优化手段，用于应对大规模数据量和高并发访问的情况。它将原本的单库单表拆分为多个数据库或表，从而分散数据存储和查询压力，提高系统的性能和可扩展性。分库分表的场景通常是在系统运行过程中，出现数据量庞大或查询性能慢等问题时引入的。

分库分表主要有两种模式：垂直拆分和水平拆分。垂直拆分是将原本的单库拆分为多个数据库，每个数据库存储特定的业务数据，如用户库、订单库等。水平拆分是将原本的单表拆分为多个表，每个表存储部分数据，如订单表拆分为 `OrderTable\_0`、`OrderTable\_1` 等。

在选择分库分表的关键因素中，需要考虑数据的访问频率、数据均匀性、业务关联性和分片键的不可变性。分片键是用于决定数据分片的字段，合理选择分片键可以提高查询性能和降低跨分片查询的开销。

对于用户数据分库分表的场景，我们根据当前用户表的数据量和每年新增用户数，预估了未来的数据量。选择了适当的分片键，并进行了分库分表的配置，保证了系统在未来较长时间内的性能和可扩展性。

我们选择了使用 ShardingSphere 来实现分库分表，因为 ShardingSphere 拥有活跃的社区，高质量的代码和丰富的功能，能够满足多样化的业务需求。根据实际情况，我们可以选择 ShardingSphere 的 JDBC 或 Proxy 模式来进行分库分表操作，根据项目的具体需求选择最适合的方式。

总的来说，分库分表是一种重要的数据库优化策略，可以帮助解决大规模数据和高并发访问带来的性能问题。在设计分库分表方案时，需要考虑系统的实际情况和未来的数据增长，合理选择分片键和分片策略，以保证系统的稳定性和可扩展性。同时，选择一个可靠的分库分表工具，如 ShardingSphere，可以帮助我们更加便捷地实现分库分表操作。

更新: 2023-10-21 13:46:03

原文: <<https://www.yuque.com/magestack/12306/pb98neetmww1rr9y>>